CSCI-4208: Developing Advanced Web Applications

Week 14: Lecture 28

React

Class Components

Overview: Class Components

- Class Components
- Inheritance & React
- Constructor & props
- Instance Variable & state
- Inherited Methods: setState
- Inherited Methods: render
- Composition & components
- Component Life Cycle

Class Components

Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML via a render() function.

Components come in two types, Class components and Function components, in this lecture we will concentrate on Class components.

Class components uses class-based syntax & an OOP approach

Inheritance & React

- The React library has a abstract class for Component.
- A class component must extend from the React.Component class so that your new component inherits all of a components built-in React functionality.
- The class component also requires a render() method, this method returns HTML.

Inheritance & React: Example

Create a Class component called Car

```
class Car extends React.Component {
  render() {
    return <h2>Hi, I am a Car!</h2>;
  }
}
```

Constructor & props

- If there is a constructor() function in your component, this function will be called when the component gets initiated.

- The constructor function is where you initiate the component's properties.

- In React, component properties should be kept in an object called state.

- The constructor function is also where you honor the inheritance of the parent component by including the super() statement, which executes the parent component's constructor function, and your component has access to all the functions of the parent component (React.Component).

Constructor & props: Example

Create a constructor function in the Car component, and add a color property:

```
class Car extends React.Component {
  constructor() {
    super();
    this.state = {color: "red"};
  }
  render() {
    return <h2>I am a {this.state.color} Car!</h2>;
  }
}
```

Constructor & props: Props

React Props are like function arguments in JavaScript and attributes in HTML.

- Props are how you pass data from one component to another, as parameters.
- Props are arguments passed into React components.
- Props are passed to components via HTML attributes.
- Props are read-only!

To send props into a component, use the same syntax as HTML attributes

Constructor & props: Example 1

Example of props in both invoker and constructor:

```
  const myelement = <Car brand="Ford" />;
```

```
class Car extends React.Component {
  constructor(props) {
    super(props);
  }
  render() {
    return <h2>I am a {this.props.brand} car!</h2>;
  }
}
```

Note: The props should always be passed to the constructor and also to the React.Component via the `super()` method.

Constructor & props: Example 2

Example when props is a variable

```
class Car extends React.Component {
  render() {
    return <h2>I am a {this.props.brand}!</h2>;
  }
}

class Garage extends React.Component {
  render() {
    const carname = "Ford";
    return (
      <div>
      <h1>Who lives in my garage?</h1>
      <Car brand={carname} />
      </div>
    );
  }
}

ReactDOM.render(<Garage />, document.getElementById('root'));
```

Instance variables & state

- React components has a built-in state object.
- The state object is where you store property values that belongs to the component.
- When the state object changes, the component re-renders.
- The state object is initialized in the constructor
- The state object can contain as many properties as you like

Instance variables & state: Examples

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = {brand: "Ford", model: "Mustang", color: "Red"};
  }
  render() {
    return (
      <div>
        <h1>My Car</h1>
      </div>
    );
  }
}
```

Instance variables & state: Accessing state

Access the state object anywhere in the component by using the syntax:

```
this.state.propertyname
```

Instance variable - state: Accessing state - Example

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = { brand: "Ford", model: "Mustang", color: "red", year: 1964 };
  }
  foo() {
    console.log(this.state.brand)
  }
  render() {
    return (
      <div>
        <h1>My {this.state.brand}</h1>
        <p>
          It is a {this.state.color}
          {this.state.model}
          from {this.state.year}.
        </p>
      </div>
    );
  }
}
```

Inherited methods: setState

To change a value in the state object, use the this.setState() method.

When a value in the state object changes, the component will re-render, meaning that the output will change according to the new value(s).

Note: Always use the setState() method to change the state object, it will ensure that the component knows its been updated and calls the render() method (and all the other lifecycle methods).

Inherited methods: setState - Example

```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = { brand: "Ford", model: "Mustang", color: "red", year: 1964 };
  }
  changeColor = () => {
    this.setState({color: "blue"});
  }
  render() {
    return (
      <div>
        <h1>My {this.state.brand}</h1>
        <p>
          It is a {this.state.color}
          {this.state.model}
          from {this.state.year}.
        </p>
        <button
          type="button"
          onClick={this.changeColor}
        >Change color</button>
      </div>
    );
  }
}
```

Inherited methods: render

- React invokes each components render method whenever it is initialized or its setState method is invoked.
- Every class component should have a render method
- render method must return the JSX (HTML-like syntax) of this component
- The returned JSX must be in a single JSX element. However, it may contain any number of child elements

Composition & Components

We can refer to components inside other components:

```
class Car extends React.Component {
  render() {
    return <h2>I am a Car!</h2>;
  }
}

class Garage extends React.Component {
  render() {
    return (
      <div>
      <h1>Who lives in my Garage?</h1>
      <Car />
      </div>
    );
  }
}


ReactDOM.render(<Garage />, document.getElementById('root'));
```

Component Lifecycle

Each component in React has a lifecycle which you can monitor and manipulate during its three main phases.

The three phases are: Mounting, Updating, and Unmounting

Component Lifecycle - Mounting

Mounting means putting elements into the DOM.

React has four built-in methods that gets called, in this order, when mounting a component:

1. `constructor()`
2. `getDerivedStateFromProps()`
3. `render()`
4. `componentDidMount()`

The `render()` method is required and will always be called, the others are optional and will be called if you define them.

Component Lifecycle - Mounting

constructor

The `constructor()` method is called before anything else, when the component is initiated, and it is the natural place to set up the initial `state` and other initial values.

The `constructor()` method is called with the `props`, as arguments, and you should always start by calling the `super(props)` before anything else, this will initiate the parent's constructor method and allows the component to inherit methods from its parent (`React.Component`).

Component Lifecycle - Mounting

getDerivedStateFromProps

The `getDerivedStateFromProps()` method is called right before rendering the element(s) in the DOM.

This is the natural place to set the `state` object based on the initial `props`.

It takes `state` as an argument, and returns an object with changes to the `state`.

Component Lifecycle - Mounting

render

The `render()` method is required, and is the method that actually outputs the HTML to the DOM.

Component Lifecycle - Mounting

componentDidMount

The `componentDidMount()` method is called after the component is rendered.

This is where you run statements that requires that the component is already placed in the DOM.

Component Lifecycle - Updating

The next phase in the lifecycle is when a component is *updated*.

A component is updated whenever there is a change in the component's `state` or `props`.

React has five built-in methods that gets called, in this order, when a component is updated:

1. `getDerivedStateFromProps()`
2. `shouldComponentUpdate()`
3. `render()`
4. `getSnapshotBeforeUpdate()`
5. `componentDidUpdate()`

The `render()` method is required and will always be called, the others are optional and will be called if you define them.

Component Lifecycle - Updating

getDerivedStateFromProps

Also at *updates* the `getDerivedStateFromProps` method is called. This is the first method that is called when a component gets updated.

This is still the natural place to set the `state` object based on the initial props.

Component Lifecycle - Updating

shouldComponentUpdate

In the `shouldComponentUpdate()` method you can return a Boolean value that specifies whether React should continue with the rendering or not.

The default value is `true`.

Component Lifecycle - Updating

render

The `render()` method is of course called when a component gets *updated*, it has to re-render the HTML to the DOM, with the new changes.

Component Lifecycle - Updating

getSnapshotBeforeUpdate

In the `getSnapshotBeforeUpdate()` method you have access to the `props` and `state` *before* the update, meaning that even after the update, you can check what the values were *before* the update.

If the `getSnapshotBeforeUpdate()` method is present, you should also include the `componentDidUpdate()` method, otherwise you will get an error.

Component Lifecycle - Updating

componentDidUpdate

The `componentDidUpdate` method is called after the component is updated in the DOM.

Component Lifecycle - Unmounting

The next phase in the lifecycle is when a component is removed from the DOM, or *unmounting* as React likes to call it.

React has only one built-in method that gets called when a component is unmounted:

- `componentWillUnmount()`

Component Lifecycle - Unmounting

componentWillUnmount

The `componentWillUnmount` method is called when the component is about to be removed from the DOM.

The End.

Coming Soon... to a Moodle near you!

Labs	React Lab
Homework	Create a React App